inzva DLSG

Homework 2

Project Overview

In this assignment, we conduct a controlled empirical study to analyse the role of **structural inductive bias** in graph neural networks (GNNs) for graph classification. The primary objective is to understand how architectural design choices — particularly the inclusion or exclusion of message passing mechanisms — influence performance across datasets with different structural and feature characteristics.

Graph classification is formally defined as learning a function

$$f : \mathcal{G} \to \mathcal{Y}$$

that maps an entire graph $G = (V, E, X)$, consisting of nodes V, edges E, and node features X, to a label $y \in \mathcal{Y}$. Unlike node-level prediction tasks, graph classification requires learning a meaningful global representation of the entire graph. Message passing neural networks address this by iteratively aggregating neighbourhood information, thereby injecting structural inductive bias into the model. In contrast, feature-only models operate on pooled node features without explicitly exploiting graph connectivity.

To systematically evaluate the impact of structural inductive bias, we compare two model families:

- **Feature-Only Models:** LinearBaseline, MLPBaseline, and DeepSets — which rely solely on permutation-invariant pooling mechanisms without neighbourhood aggregation.
- **Message Passing Models:** GCN, GraphSAGE, GAT, and GIN — which explicitly incorporate graph topology via neighbourhood aggregation.

The experimental evaluation spans both molecular and non-molecular graph benchmarks. Specifically, we evaluate:

- **ogbg-molhiv** (binary molecular classification, evaluated with ROC-AUC),
- **PROTEINS**, **IMDB-MULTI**, and **REDDIT-BINARY** (evaluated with accuracy).

These datasets differ substantially in graph size, feature dimensionality, and semantic meaning of edges. Molecular datasets contain chemically meaningful edges and rich node features, whereas social network datasets often rely more heavily on structural patterns. This diversity allows us to isolate when structural inductive bias is beneficial and when simple feature aggregation is sufficient.

To ensure fairness and reproducibility, we use official dataset splits (for OGB), run at least three random seeds per configuration, and report results as mean $\pm$ standard deviation. All models are compared under controlled hyperparameter settings, including hidden dimension, number of layers, aggregation and readout strategies, activation functions, regularisation techniques, and total parameter count.

The central research question of this study is:

> Does increasing structural inductive bias and model expressivity consistently improve graph classification performance, or is the benefit dataset-dependent?

Figure 1: Training and validation curves for the PROTEINS dataset using GCN (mean pooling, hidden dimension = 64, 4 layers) across three different random seeds. The left panels show train and validation loss, while the right panels show validation ROC-AUC over epochs. We observe relatively stable training behaviour with moderate fluctuations across seeds. Early stopping is triggered when validation performance saturates. The best validation ROC-AUC varies across seeds, demonstrating the importance of reporting mean $\pm$ standard deviation.

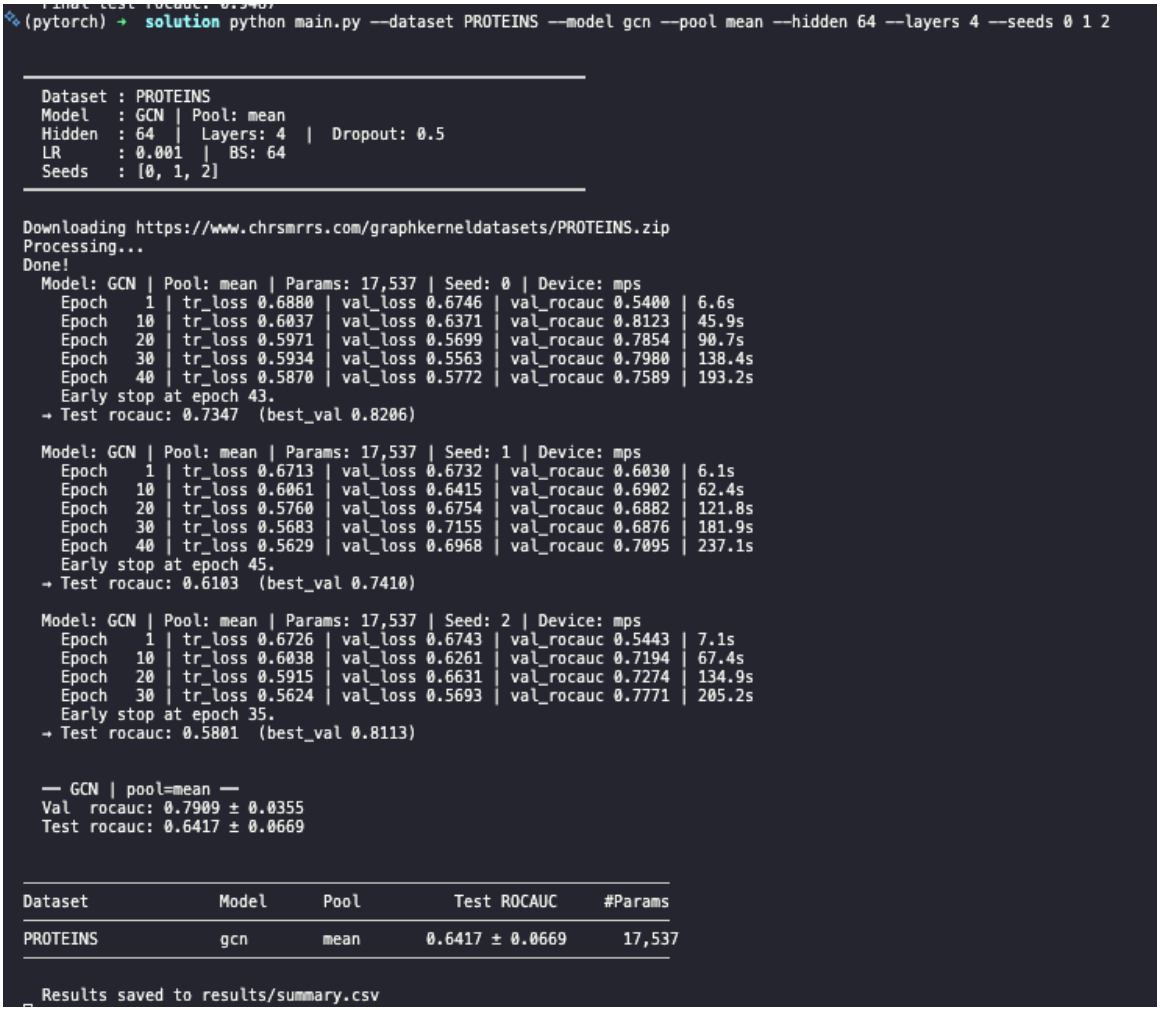
We further analyse trends such as overfitting, oversmoothing, and the trade-off between model complexity and generalisation.

Final Note to Students. You must complete the full report following the provided structure. However, you are free to customise sections, introduce additional analyses, and design your own experimental variations. You are not required to strictly use the provided codebase, although it already provides a well-structured implementation to experiment efficiently with the requested setup.

Another Final Note to Students. It might be hard to understand or fully grasp the concept at first. Do not try to understand everything completely, since it is a very broad topic and it can take a lot of time. Focus on the datasets and try to understand them. Create `.ipynb` notebooks, perform EDA on each dataset, and visualise them to see what they look like. I encourage you to talk with LLMs about these problems. You should ask your questions, even the stupidest ones. Use Socratic questioning to better grasp the subject at hand. For example, go and ask: "What is structural inductive bias?" or search about the datasets. Ask about message passing. What is the difference between GCN and GraphSAGE or GAT?

I believe we should be able to learn anything and use that knowledge to produce some output. I believe that is the real outcome you should gain from this homework, apart from learning about GNNs. I think this is where the world is headed: we should grasp things very quickly and, using that glimpse of knowledge, turn it into something meaningful. However, the philosophical question is: Do we have to be like that? Do we have to turn into robots that consume knowledge and force ourselves to produce something meaningful?

I think not. I believe we should also write poems and eat pizza and lahmacun. However, my beliefs are not that important. So, please complete the code and the reports.



```
(pytorch) → solution python main.py --dataset PROTEINS --model gcn --pool mean --hidden 64 --layers 4 --seeds 0 1 2

  ────────────────────────────────────────────────────────────
    Dataset : PROTEINS
    Model   : GCN | Pool: mean
    Hidden  : 64  |  Layers: 4  |  Dropout: 0.5
    LR      : 0.001  |  BS: 64
    Seeds   : [0, 1, 2]
  ────────────────────────────────────────────────────────────

  Downloading https://www.chrsmrrs.com/graphkerneldatasets/PROTEINS.zip
  Processing...
  Done!
    Model: GCN | Pool: mean | Params: 17,537 | Seed: 0 | Device: mps
      Epoch    1 | tr_loss 0.6880 | val_loss 0.6746 | val_rocauc 0.5400 | 6.6s
      Epoch   10 | tr_loss 0.6037 | val_loss 0.6371 | val_rocauc 0.8123 | 45.9s
      Epoch   20 | tr_loss 0.5971 | val_loss 0.5699 | val_rocauc 0.7854 | 90.7s
      Epoch   30 | tr_loss 0.5934 | val_loss 0.5563 | val_rocauc 0.7980 | 138.4s
      Epoch   40 | tr_loss 0.5870 | val_loss 0.5772 | val_rocauc 0.7589 | 193.2s
      Early stop at epoch 43.
    → Test rocauc: 0.7347  (best_val 0.8206)

    Model: GCN | Pool: mean | Params: 17,537 | Seed: 1 | Device: mps
      Epoch    1 | tr_loss 0.6713 | val_loss 0.6732 | val_rocauc 0.6030 | 6.1s
      Epoch   10 | tr_loss 0.6061 | val_loss 0.6415 | val_rocauc 0.6902 | 62.4s
      Epoch   20 | tr_loss 0.5760 | val_loss 0.6754 | val_rocauc 0.6882 | 121.8s
      Epoch   30 | tr_loss 0.5683 | val_loss 0.7155 | val_rocauc 0.6876 | 181.9s
      Epoch   40 | tr_loss 0.5629 | val_loss 0.6968 | val_rocauc 0.7095 | 237.1s
      Early stop at epoch 45.
    → Test rocauc: 0.6103  (best_val 0.7410)

    Model: GCN | Pool: mean | Params: 17,537 | Seed: 2 | Device: mps
      Epoch    1 | tr_loss 0.6726 | val_loss 0.6743 | val_rocauc 0.5443 | 7.1s
      Epoch   10 | tr_loss 0.6038 | val_loss 0.6261 | val_rocauc 0.7194 | 67.4s
      Epoch   20 | tr_loss 0.5915 | val_loss 0.6631 | val_rocauc 0.7274 | 134.9s
      Epoch   30 | tr_loss 0.5624 | val_loss 0.5693 | val_rocauc 0.7771 | 205.2s
      Early stop at epoch 35.
    → Test rocauc: 0.5801  (best_val 0.8113)


    ── GCN | pool=mean ──
    Val  rocauc: 0.7909 ± 0.0355
    Test rocauc: 0.6417 ± 0.0669


  ──────────────────────────────────────────────────────────────
  Dataset              Model      Pool          Test ROCAUC     #Params
  ──────────────────────────────────────────────────────────────
  PROTEINS             gcn        mean       0.6417 ± 0.0669      17,537
  ──────────────────────────────────────────────────────────────

    Results saved to results/summary.csv
```

Figure 2: This is what one of the experiments should look like after you have implemented everything correctly.

Delivery Instructions

You are required to submit both:

- **Source Code:** The complete implementation used for your experiments, including training scripts, model definitions, and any additional utilities required to reproduce your results. The code must run without modification and reproduce the reported results when executed with the specified settings. You can create a `.txt` file and put your recorded experiment's CLI commands
- **Final Report:** The report must include dataset analysis, methodological details, experimental setup, quantitative results (mean $\pm$ standard deviation), figures, and a discussion of findings. You can find the report here (←click). You do not have to use Overleaf. You can choose your own favorite text editor. However, I strongly recommend.